

Get the farmer, wolf, goat and cabbage across the river

Programming and Tools for AI – Week 6

James McDermott



Part 1: the problem

The problem

A farmer must ferry a wolf, a goat and a cabbage across a river in a boat that holds the farmer and one other thing. Left unsupervised, the wolf would eat the goat, or the goat would eat the cabbage.

Find the shortest sequence of crossings.

It is at least twelve hundred years old:
Alcuin of York described it for
Charlemagne's court (8th Century).



The Ormesby Psalter, c. 1310. Bodleian Library,
public domain.

Why?

It's a classic problem in Good Old-Fashioned AI, and we will learn a lot that generalises. The same algorithm solves many different problems:

- **word ladders:** HEAD - HEAL - TEAL - TELL - TALL - TAIL
- **routing:** Google Maps
- Rubik's cube
- Paths in social networks



“Six degrees of separation” in a social network

Moving around a graph

1. A **state** represents how things are at a moment in time in the game.
2. A **move** takes you from one state to another.
3. States and moves make a **graph**, and the answer is a **path** through it.

All of these are problem-independent!

Defining states in the FWGC problem

We will agree the ordering (*farmer, wolf, goat, cabbage*).

The state needs to represent the **locations** at a point in time, eg:

START = "LLLL" # (*F, W, G, C*) all on the left bank

GOAL = "RRRR" # (*F, W, G, C*) all on the right bank

So a state is a four-character string. There are $2 \times 4 = 16$ states.

Our state representation is short and hashable.

Check we understand

Remember the order: (*farmer, wolf, goat, cabbage*).

- Is this state allowed? LRLR
- Is this state allowed? LLRR
- Is this state allowed? RLRR
- Is this move allowed? LLLL → RLRL
- Is this move allowed? LLLL → LLLR

Sixteen states, ten of them safe

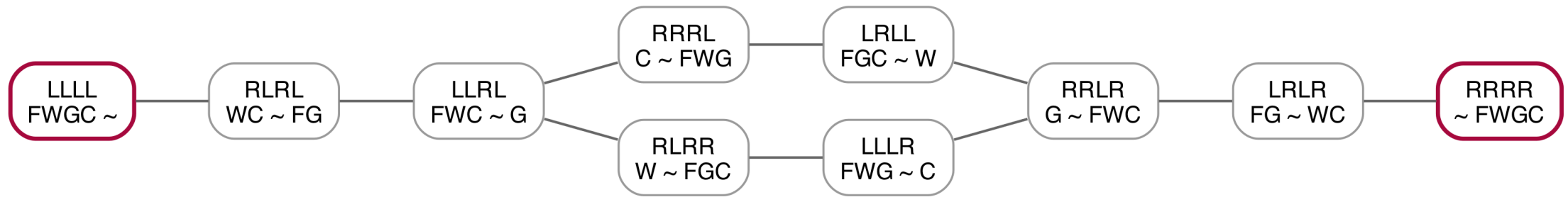
LLLL FWGC ~	LLLR FWG ~ C	LLRL FWC ~ G	LLRR FW ~ GC
LRLl FGC ~ W	LRLR FG ~ WC	LRRL FC ~ WG	LRRR F ~ WGC
RLLL WGC ~ F	RLLR WG ~ FC	RLRL WC ~ FG	RLRR W ~ FGC
RRLL GC ~ FW	RRLR G ~ FWC	RRRL C ~ FWG	RRRR ~ FWGC

Six of the sixteen are non-safe, ie something gets eaten. `python E01_river.py --states` prints the same table.

Is a state safe?

```
def is_safe(state):  
    farmer, wolf, goat, cabbage = state # eg 'LRRL'  
    if goat == wolf and farmer != goat:  
        return False  
    if goat == cabbage and farmer != goat:  
        return False  
    return True
```

The whole puzzle, as a graph



Ten safe states and ten state-to-state moves. Start at LLLL and try to find RRRR. Every edge in the graph is a river crossing.

The graph as a dict

```
RIVER_GRAPH = {  
    "LLLL": ["RLRL"],  
    "RLRL": ["LLLL", "LLRL"],  
    "LLRL": ["RLRL", "RRRL", "RLRR"],  
    "RRRL": ["LLRL", "LRLL"],  
    "LRLL": ["RRRL", "RRLR"],  
    "RLRR": ["LLRL", "LLLR"],  
    "LLLR": ["RLRR", "RRLR"],  
    "RRLR": ["LRLL", "LLLR", "LRLR"],  
    "LRLR": ["RRLR", "RRRR"],  
    "RRRR": ["LRLR"],  
}
```

This is an **adjacency dict**: each key is a node and its value is the list of nodes it joins to. Every edge appears twice: eg LLLL → RLRL means “farmer and goat go right” and RLRL → LLLL means “farmer and goat go left”.

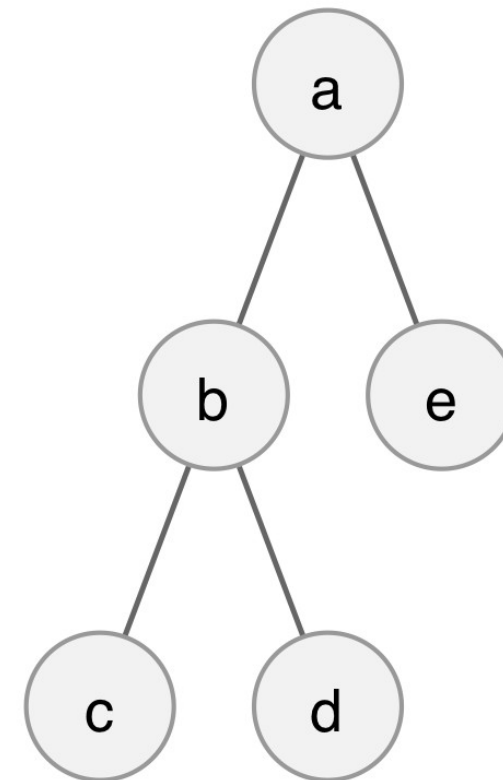
Part 2: searching the graph

Searching the graph

You may have heard of two graph traversal algorithms:

- **Depth-first:** follow one path in the graph as far as it goes. Every time we reach a dead end, go back to the last turn we didn't try. We used this with the bicycle bill of materials.
- **Breadth-first:** look at every state one step away, then every state two steps away, and so on.

Depth-first finds **a** path. Breadth-first finds the **shortest** path.



Taking the left child first, depth-first visits a b c d e. Breadth-first visits a b e c d.

Breadth-first search: need three containers

The frontier. States we have reached but not yet looked at. We use a **queue** to give first in, first out (FIFO). This guarantees we try all nodes at n steps before any at $n+1$ steps.

The “came from” data. For each state we have reached, need to know the best path to that state. Store it as a **dict** `{state: predecessor_state}`.

The visited set. The set of states we have already reached, so we can avoid revisiting. But notice: **came_from** already stores this information. We can drop this.

Queues (first-in, first-out)



A queue is a **first-in, first-out** structure: we append on the right, and de-queue (remove for processing) from the left. Items leave in the order they arrived.

A list is not a good queue

In Python, a queue is provided as `collections.deque`, short for “double-ended queue”. It is basically a list, except that removing element 0 is efficient, **$O(1)$** .

```
L = [0, 1, 2] # a list is not a good queue:
L.append(4) #  $O(1)$ 
next_item = L[0] #  $O(1)$ 
del L[0] #  $O(n)$  <- inefficient!
```

```
from collections import deque
q = deque([0, 1, 2])
q.append(4) #  $O(1)$ 
next_item = q.popleft() #  $O(1)$  <- efficient!
```

Breadth-first search, in Python

```
def bfs_graph(graph, start, goal):  
    frontier = deque([start])  
    came_from = {start: None}  
    while frontier: # while some nodes are still in the queue  
        state = frontier.popleft() # take the first node from q  
        if state == goal:  
            return path_to(came_from, state)  
        for nxt in graph[state]: # for all nodes adjacent to state  
            if nxt not in came_from: # no revisiting  
                # shortest path to nxt comes from state  
                came_from[nxt] = state  
                frontier.append(nxt)  
    return None
```

We dropped the visited set

The `came_from` dict is doing two jobs:

- The key-value pairs record which state we reached each state *from*.
- The keys are the set of states we have reached, ie the visited set.

The keys are strings, which are hashable.

Recall that `in` on a dict is $O(1)$.

States must be hashable

came_from is a dict, so every state is a dict key, so every state must be hashable.

```
"LLRL"           # fine: strings are hashable
(0, 3, 1, 2)      # fine: tuples of hashables are too
[0, 3, 1, 2]      # TypeError: unhashable type: 'list'
{"farmer": "L"}   # TypeError
```

If your state is a list just convert it to a tuple, eg: tuple(state).

If your state is a list of lists, like a chess board, just convert it to a tuple or tuple of tuples, eg: tuple(tuple(row) for row in board)

Reconstructing the path

It's not enough to return True when we reach the goal. We need to see the path we found. `came_from` stores the path. We just retrace our steps.

```
def path_to(came_from, state):  
    path = [state]  
    while came_from[path[-1]] is not None:  
        path.append(came_from[path[-1]])  
    path.reverse()  
    return path
```

The answer

```
bfs_graph(RIVER_GRAPH, 'LLLL', 'RRRR')
```

The trick is the fourth move, which takes the goat “the wrong way”!

FWGC ~

WC ~ FG farmer takes the goat -->

FWC ~ G <-- farmer rows back alone

C ~ FWG farmer takes the wolf -->

FGC ~ W <-- farmer takes the goat

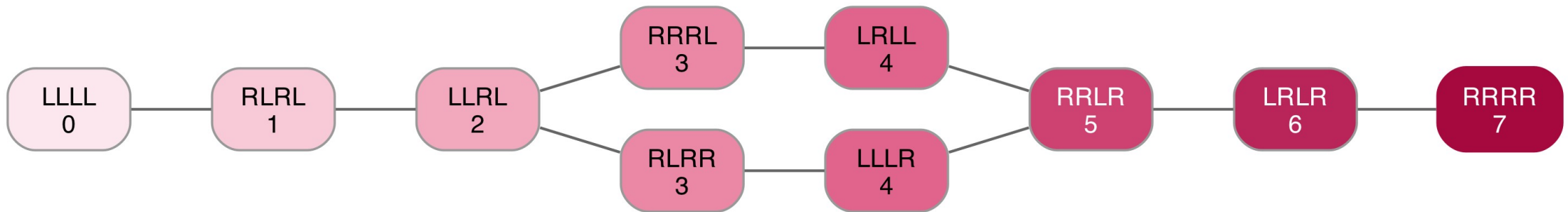
G ~ FWC farmer takes the cabbage -->

FG ~ WC <-- farmer rows back alone

~ FWGC farmer takes the goat -->

7 crossings

The frontier, ring by ring



How many crossings each state is from the start. BFS finishes a whole ring before it starts the next.

Why BFS finds the shortest path

Claim: when BFS first reaches a state, it has reached it by the shortest route (or **a** shortest route, if there are multiple equally-short routes).

Sketch: the frontier is a queue, so states leave it in the order they entered, and each state's neighbours are added just after it. So the queue holds states at distance d , followed by states at distance $d + 1$, and never the other way round. A state first seen at distance d cannot be reached in fewer than d steps, or we would have seen it earlier.

That argument depends on the assumption: **every move costs the same**. Later we will see an algorithm for edges with different costs.

Computational complexity of BFS

Every state comes off the frontier at most once, because we mark it visited the moment we first see it. For each one, we look at each of its edges once.

$O(V + E)$

where V is the number of states and E the number of moves. That is **linear** in the size of the graph, which is as good as can be.

Part 3: the graph we never build

We typed the FWGC graph out by hand

problem	states
farmer, wolf, goat, cabbage	10
word ladder, 4-letter words	4,360
15-puzzle	10,461,394,944,000
Rubik's cube	4.3×10^{19}

Typing out ten states by hand was tedious. For large graphs it would be impossible.

Compute neighbours when needed instead of storing them

We do not need the whole graph. At any moment BFS needs the neighbours of **one** state.

So replace the dict with a function:

```
graph[state]          # look up the neighbours  
successors(state)    # calculate the neighbours when needed
```

This is the same concept as **laziness** which we previously met in the context of generators.

The successor function

```
def successors(state):
    farmer, wolf, goat, cabbage = state
    there = "R" if farmer == "L" else "L"
    crossings = [there + wolf + goat + cabbage] # F cross
    if wolf == farmer: # FW cross
        crossings.append(there + there + goat + cabbage)
    if goat == farmer: # FG cross
        crossings.append(there + wolf + there + cabbage)
    if cabbage == farmer: # FC cross
        crossings.append(there + wolf + goat + there)
    for crossed in crossings:
        if is_safe(crossed): # only safe crossings
            yield crossed
```

BFS with a generative graph

```
def bfs(successors, start, goal):
    frontier = deque([start])
    came_from = {start: None}
    while frontier:
        state = frontier.popleft()
        if state == goal:
            return path_to(came_from, state)
        for nxt in successors(state):      # <-- was: graph[state]
            if nxt not in came_from:
                came_from[nxt] = state
                frontier.append(nxt)
    return None
```

The graph is now specified by a **function**, successors. If we have a graph stored as a dict, we can still use the new version bfs, like this:

```
bfs(lambda s: RIVER_GRAPH[s], START, GOAL)
```

An infinite graph

“A calculator has only 3 buttons. The first multiplies the current value by 3, the second adds 2 and the third subtracts 2. Starting with 0 what is the least number of presses you need to reach 100?”

– <https://puzzling.stackexchange.com/questions/91090/three-button-calculator>

We can solve this just by writing a new `successors` function.

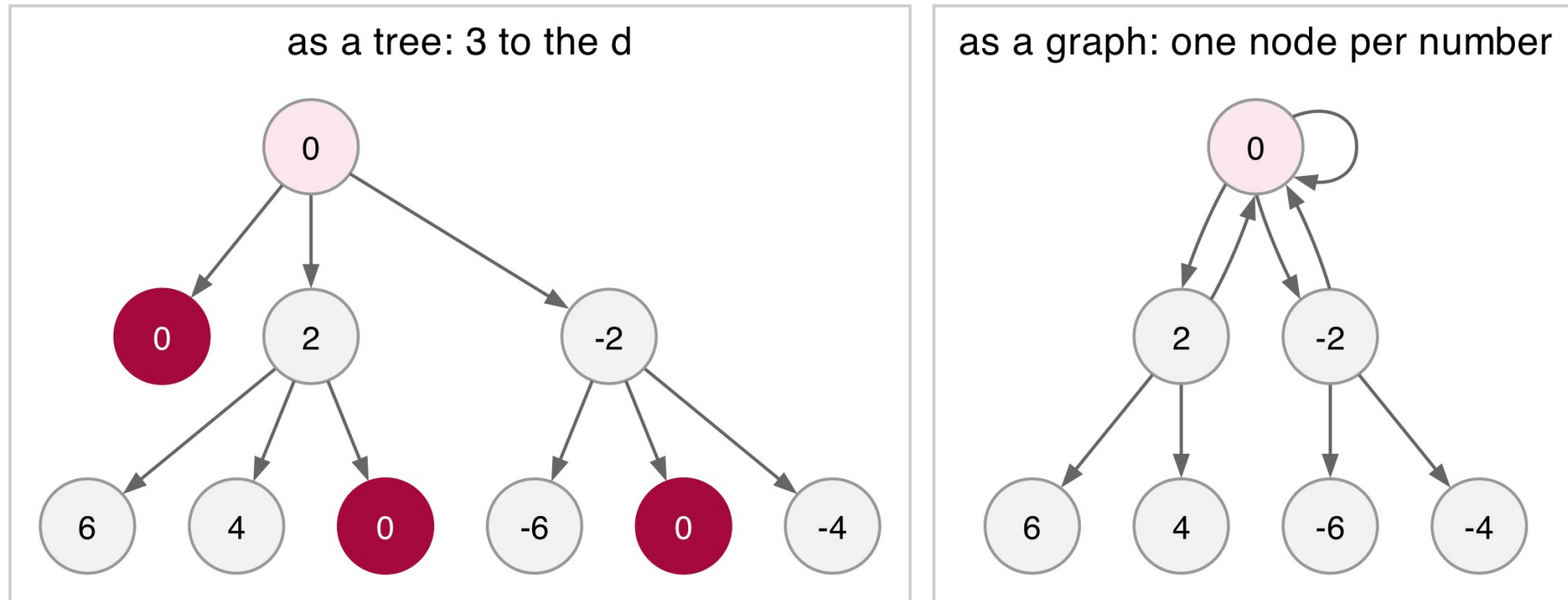
An infinite graph

```
def calc_successors(n):  
    yield n * 3  
    yield n + 2  
    yield n - 2
```

```
>>> bfs(calc_successors, 0, 100)  
[0, 2, 4, 12, 36, 34, 102, 100]
```

It took 7 presses. There is no largest integer, so this graph is infinite. Note the state is the **number on the display**, not the buttons pressed so far.

The 3-button calculator as tree and as graph



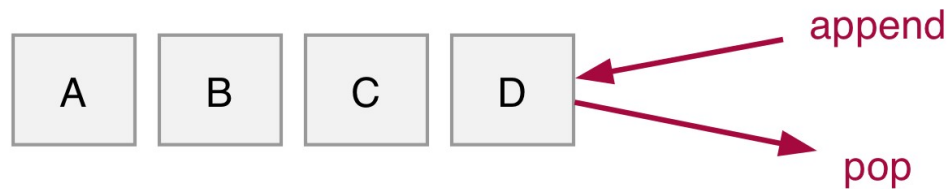
Every route to a number is a different branch of the tree. Tree size grows as 3 to the depth. With a visited set, we can stop when hitting a known number.

Queue, or stack

queue: first in, first out



stack: last in, first out



These data structures differ only in which end you take an item from.

Depth-first search: one word changes

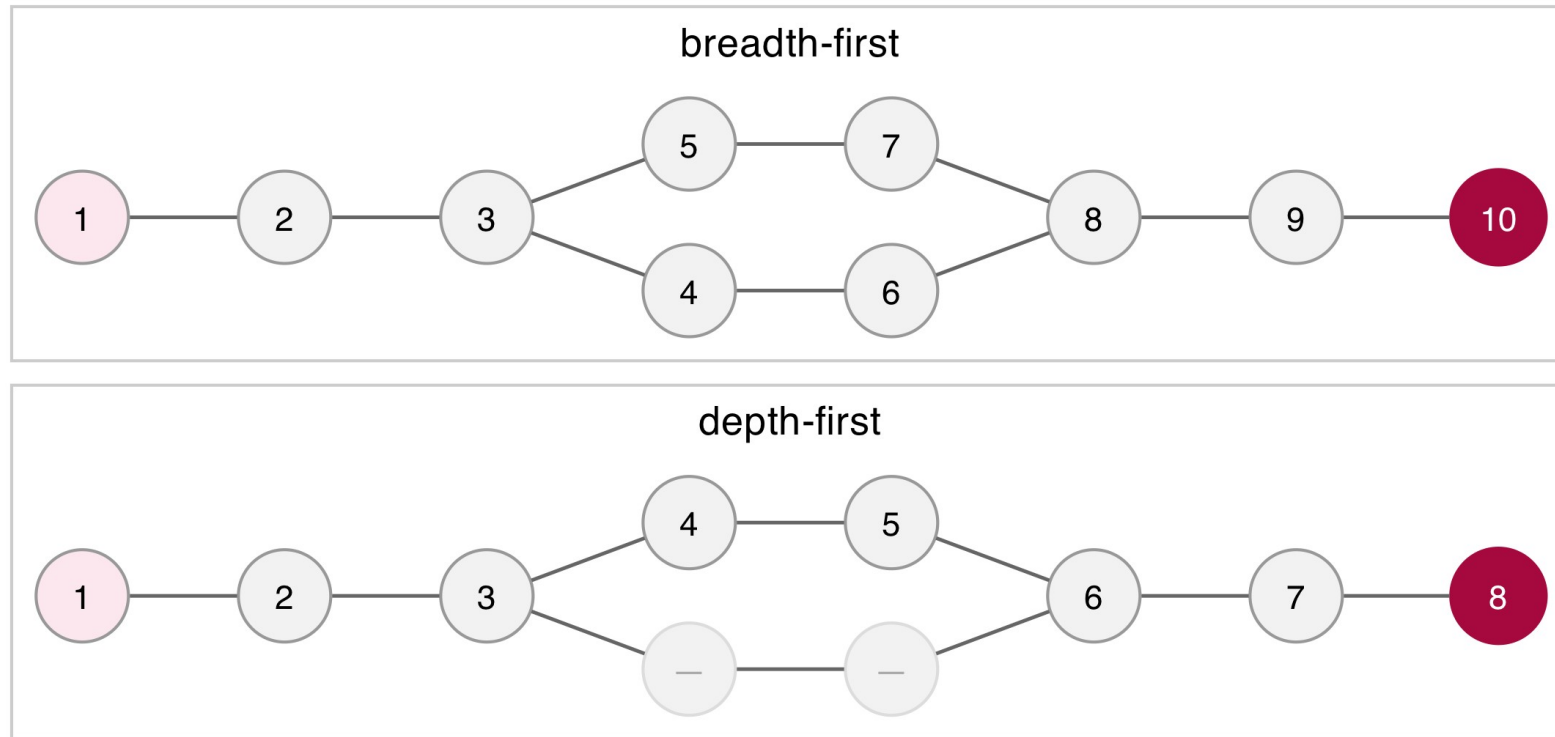
```
state = frontier.pop()           # was: frontier.popleft()
```

`popleft` takes the state that has been waiting longest — a **queue**.

`pop` takes the one that arrived most recently — a **stack**.

That change is the only difference between breadth-first and depth-first search.

The same graph in two different orders



The order each search takes states off its frontier. Breadth-first pops all ten. Depth-first reaches the goal after eight and never pops the two greyed ones: they were pushed onto the stack, but the branch pushed *after* them ran straight to the goal, and a stack hands back the newest first.

Recursion uses an implicit stack

Week 3's `parts(node)` walked the bicycle bill of materials tree depth-first, but the algorithm had no stack. Why not?

```
def parts(node):  
    if "children" not in node:  
        yield node  
    else:  
        for c in node["children"]:  
            yield from parts(c)
```

Here, Python effectively keeps the stack for us, internally, as the **call stack**.

DFS with a stack (non-recursive)

```
def dfs(successors, start, goal):  
    frontier = [start] # list  
    came_from = {start: None}  
    while frontier:  
        state = frontier.pop()  
        if state == goal:  
            return path_to(came_from, state)  
        for nxt in successors(state):  
            if nxt not in came_from:  
                came_from[nxt] = state  
                frontier.append(nxt)  
    return None
```

BFS or DFS?

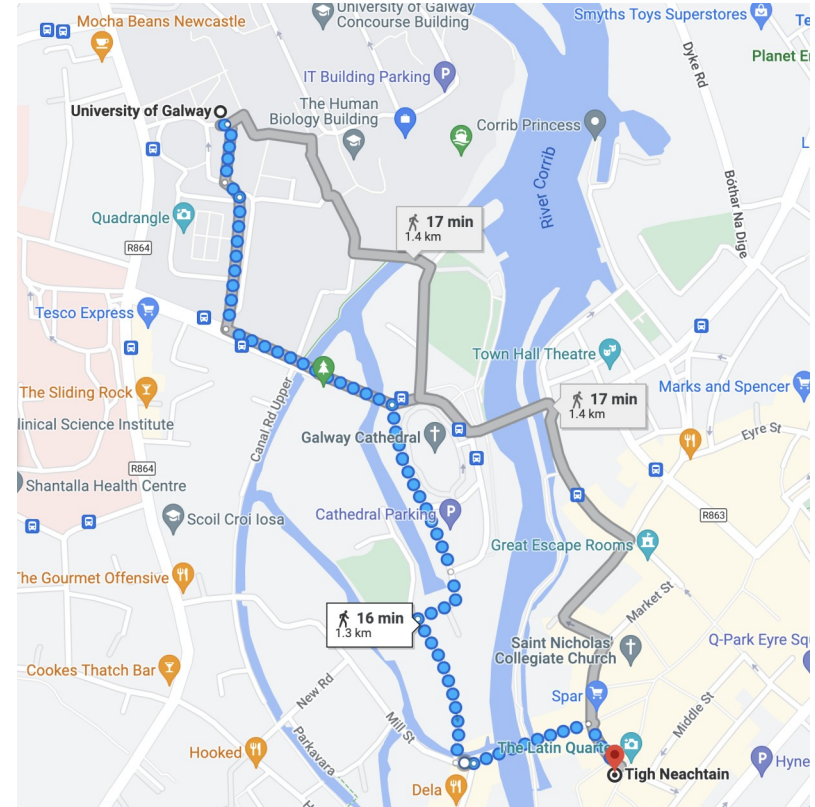
	BFS	DFS
container	queue (deque)	stack (list)
finds	shortest path	a path
memory	whole frontier, can be huge	one branch deep
infinite graph	safe	can walk away forever

Use BFS when searching for a shortest path. DFS is mostly for traversal (eg, the bicycle bill of materials) and cycle-detection.

Part 4: when the moves have costs

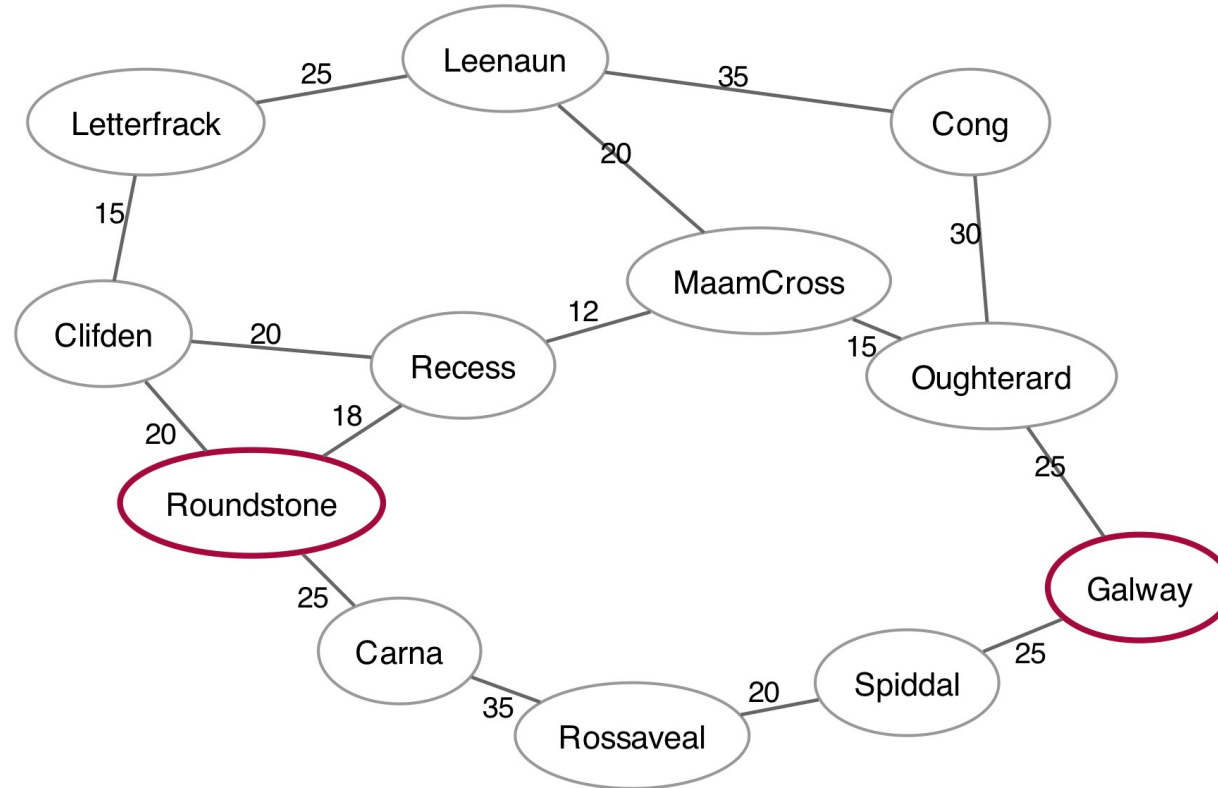
Routing problems

Google Maps is graph search. The nodes are junctions, the edges are road segments, and the weight on an edge is how long that road segment takes.



University of Galway to Tigh Neachtáin. Google Maps.

Traversing a map



Connemara, with approx. driving times in minutes, represented as edge **weights**.

Fewest roads versus fastest

```
$ python E01_river.py --route Galway Roundstone
```

fewest roads: 4 roads, 105 min

Galway → Spiddal → Rossaveal → Carna → Roundstone

quickest: 4 roads, 70 min

Galway → Oughterard → MaamCross → Recess → Roundstone

If every road segment took the same length of time, these two routes would be equally good. But in reality, some road segments are longer than others. How to make an algorithm find the faster one?

Why BFS is not suitable when edges have costs

Part 2 claimed: the first time BFS reaches a state, it has reached it by a shortest route. The proof assumed **every move costs the same**.

When weights differ, a three-edge route could be cheaper than a two-edge one, but BFS would prefer the two-edge route.

We need a different rule for **which state to expand next**.

Dijkstra's algorithm

- BFS: of all unfinished states we can reach, choose the one we found **first**.
- Dijkstra: of all unfinished states we can reach, choose the one we can reach **most cheaply**.

So we need a frontier data structure that gives us the cheapest item rather than the oldest one. That is we need a **priority queue**.

```
pq = PriorityQueue()           # provided
pq.push("Roundstone", 70)     # add (item, priority)
node, cost = pq.pop()         # get the cheapest item in the queue
len(pq)                       # now zero
```

As in Part 3, the graph must be provided as a **function**. ROADS is a dict of dicts, so we use `lambda t: ROADS[t].items()`.

Dijkstra's algorithm

```
def dijkstra(successors, start, goal):  
    frontier = PriorityQueue() # create the queue  
    frontier.push(start, 0) # ... add start node ...  
    came_from = {start: (None, 0)} # ... same for came_from  
    while frontier: # as long as there are known nodes left  
        node, cost = frontier.pop() # get the next node  
        if node == goal:  
            # we are finished; retrace our steps  
            steps = {w: prev for w, (prev, _) in came_from.items()}  
            return cost, path_to(steps, node)  
        for nxt, weight in successors(node): # for each neighbour  
            # if nxt is new OR this path to nxt is an improvement  
            if (nxt not in came_from  
                or cost + weight < came_from[nxt][1]):  
                frontier.push(nxt, cost + weight) # add to queue  
                came_from[nxt] = (node, cost + weight) # add
```

Why came_from has to remember more in Dijkstra

BFS stores only *which state we came from*:

```
if nxt not in came_from:                # BFS
```

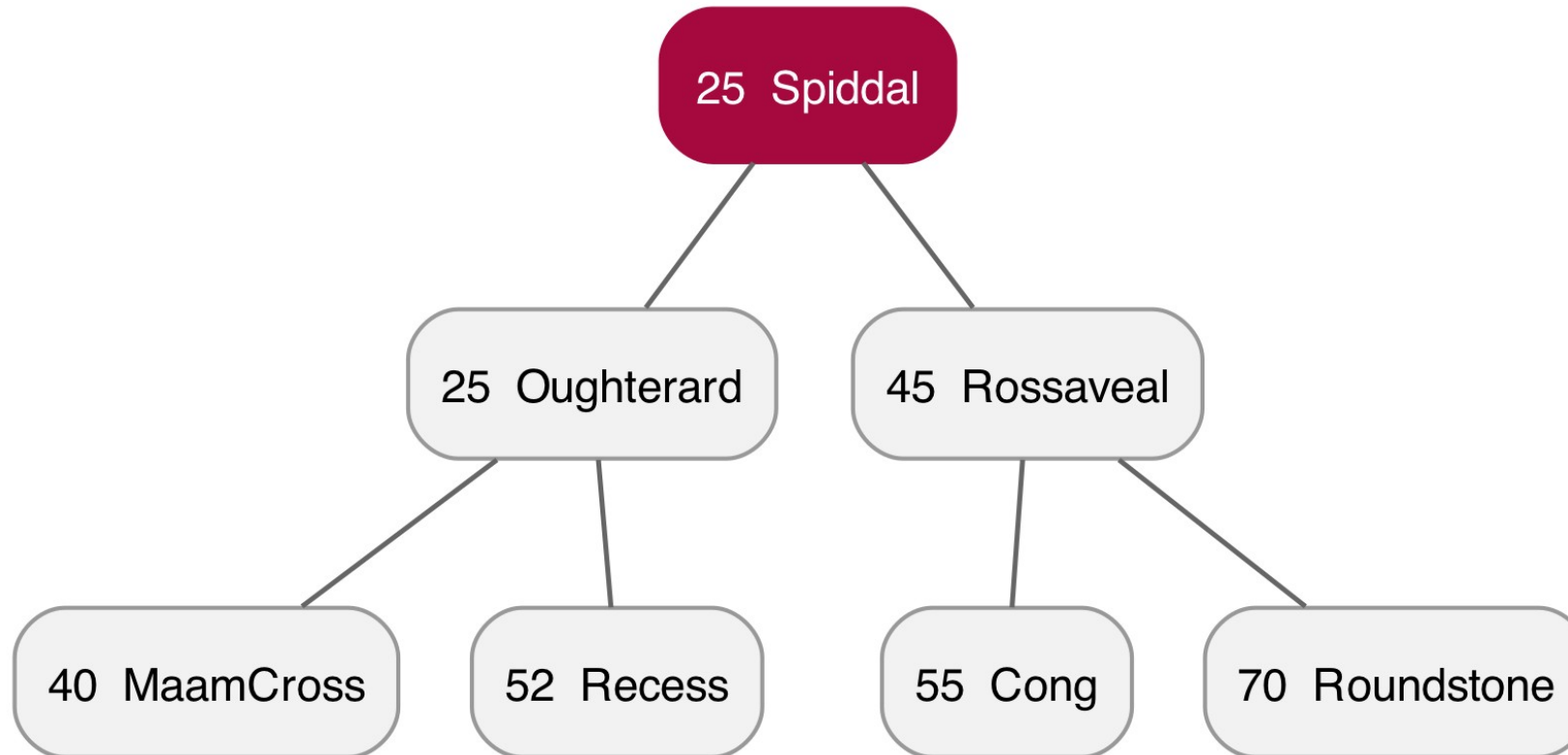
That's ok. Every move costs the same, so the first route to a state is the best one.

In Dijkstra, moves have different costs. A route discovered later in the loop can be cheaper. So we keep the route **and the cost**.

```
if (nxt not in came_from  
    or cost + weight < came_from[nxt][1]):
```

That one test does two jobs: it lets a better route in, and it rejects a worse one.

Inside the priority queue (not examinable)



One page on what `PriorityQueue` keeps in `self._heap`. A parent is never worse than its children — 45 sits above 40 — which is enough to put the smallest item at the root, and cheap enough that push and pop cost $O(\log n)$ instead of a full sort.

Computational complexity

	BFS	Dijkstra
next state	first found	cheapest found
container	queue	heap
minimises	number of edges	total weight
cost	$O(V + E)$	$O((V + E) \log V)$

The $\log V$ is because of the heap (inside the priority queue). Dijkstra with all weights equal to 1 *is* BFS. The Dijkstra complexity is **not examinable**.

Meeting in the middle

BFS or Dijkstra explores an expanding “ball” around the start. With branching factor b , a ball of radius d holds about b^d states.

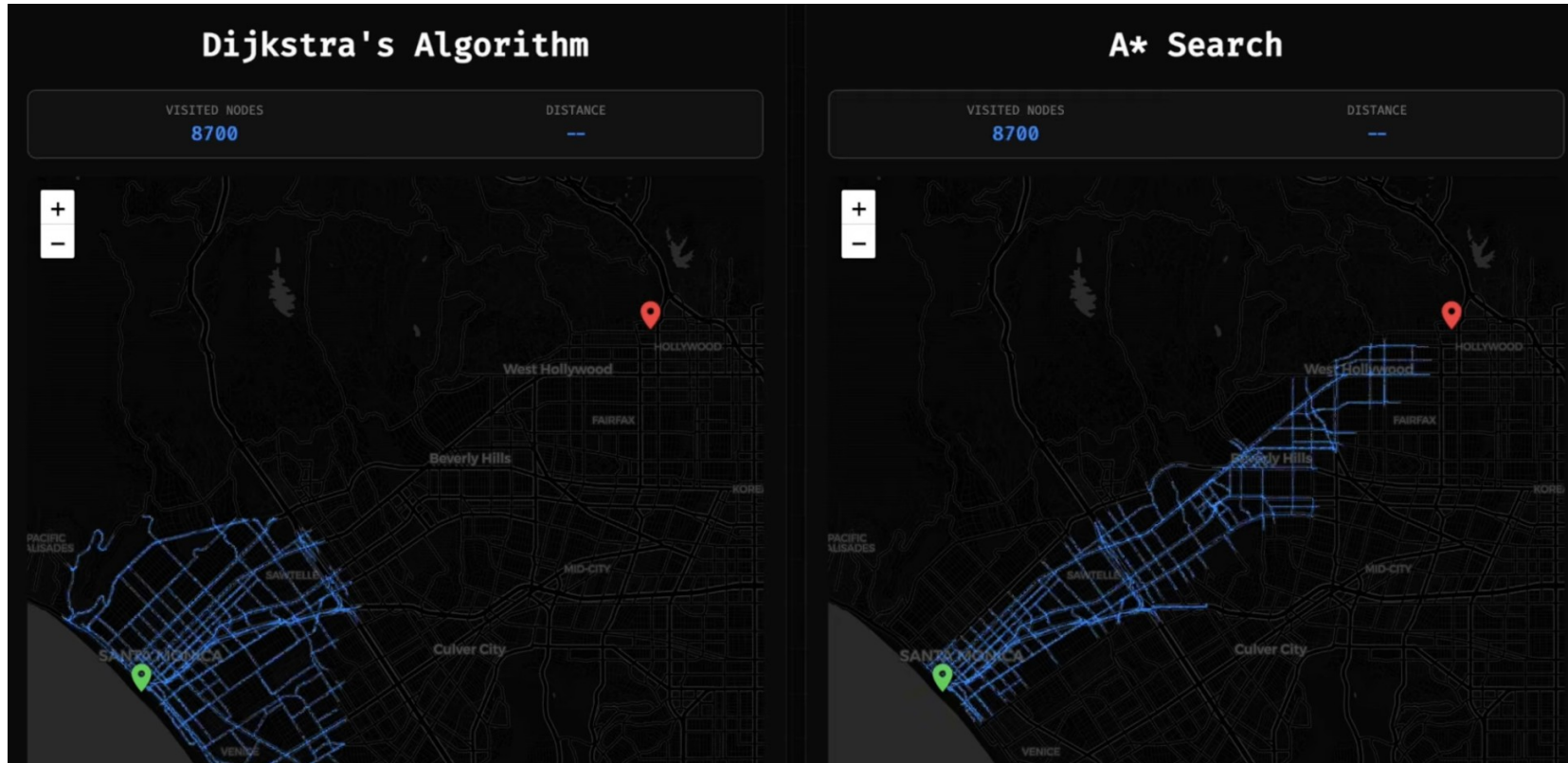
In **Bidirectional Dijkstra**, instead we search **from both ends at once** and stop where the two balls touch. Two balls of radius $d/2$ each give about $2b^{d/2}$ states.

A map is flat, so half the radius is a quarter of the area: **about half the work**. On a puzzle graph, where the ball really does grow as b^d , it saves far more.



Two frontiers growing towards each other.
youtube.com/shorts/80M4By_-ztg

A* Search (not examinable)



BFS and Dijkstra algorithms are **blind**. They don't know which direction the goal is in, so they search in a ball, expanding uniformly in every direction. In **A* Search**, instead we **expand the state that looks most promising**. It can be much more efficient, as shown. Dijkstra spreads in every direction; A* leans towards the goal. [youtube.com/watch?v=AA-sRda-6WY](https://www.youtube.com/watch?v=AA-sRda-6WY)

Summary

- Searching for best path to a given destination in a **graph** composed of states and moves.
- **BFS** finds the shortest path. Use a queue for the frontier. Track the visited states to avoid revisiting and to retrace steps at the end.
- **DFS** is the same code as BFS, but with pop instead popleft — a stack, not a queue.
- The graph does not have to exist. A **successor function** computes neighbours on demand: the only way to search 10^{19} states.
- States must be **hashable**.
- **Dijkstra** is BFS with a **priority queue**: take the cheapest state next, not the nearest.

New this week: deque with append / popleft / pop; a PriorityQueue class of our own wrapping heapq, with `__len__` so while frontier: works; a **successor function** passed as an argument, and lambda to make a dict look like one; dicts as predecessor pointers; set as a visited set.